



Ecole Supérieure Informatique Sidi Bel Abbès
Second Cycle

RÉALITÉ VIRTUELLE ET AUGMENTÉE TP 4 : Simulation de Caméra et Raycast

Durée estimée : 2h

0.1 Objectifs du TP

1. Comprendre comment une caméra “regarde” une cible (LookAt).
2. Implémenter un système de caméra de surveillance (Tracking simple).
3. Simuler la “vision” par Raycasting.
4. Interagir avec des objets par le regard (Gaze-based interaction).

0.2 Partie 1 : Caméra de Surveillance

Dans cette partie, nous allons créer une caméra qui suit le joueur du regard, simulant le tracking d'un objet.

0.2.1 1. Mise en Place

1. Créez une nouvelle scène **SecurityCamera**.
2. Créez un **Plane** (Sol) et un **Cube** (Le Joueur). Nommez-le “Player”.
3. Ajoutez un script **PlayerMovement** simple pour bouger le cube avec les flèches (ou réutilisez vos connaissances).
4. Créez un objet vide “SecurityCam_Base” en hauteur ($Y = 5$) dans un coin.
5. Ajoutez un enfant **Cylinder** (renommé “Lens”) couché horizontalement pour ressembler à une caméra.

0.2.2 2. Script LookAt

Créez un script **LookAtTarget.cs** et attachez-le à votre caméra de sécurité :

```
1 using UnityEngine;
2
3 public class LookAtTarget : MonoBehaviour
4 {
5     public Transform target; // La cible à suivre (Le Player)
6     public float speed = 5f; // Vitesse de rotation (Damping)
7
8     void Update()
9     {
10         if (target == null) return;
11
12         // 1. Calculer la direction vers la cible
13         Vector3 direction = target.position - transform.position;
14
15         // 2. Créer la rotation qui regarde dans cette direction
16         Quaternion targetRotation = Quaternion.LookRotation(direction);
17
18         // 3. Appliquer la rotation doucement (Slerp = Spherical Linear
19         //    Interpolation)
20         transform.rotation = Quaternion.Slerp(transform.rotation,
21         targetRotation, speed * Time.deltaTime);
```

```
20 }  
21 }
```

Assignez le “Player” dans le champ Target. Lancez le jeu et bougez le cube. La caméra doit vous suivre.

0.3 Partie 2 : Le Raycast (Vision Laser)

La caméra nous voit-elle vraiment ? Pour le savoir, nous allons lancer un rayon invisible.

0.3.1 1. Le Script de Vision

Créez un script `SecurityVision.cs` sur la caméra.

```
1 using UnityEngine;  
2  
3 public class SecurityVision : MonoBehaviour  
4 {  
5     public float viewDistance = 20f;  
6     public LayerMask obstaclesLayer; // Ce qui bloque la vue (Murs)  
7     public Color alertColor = Color.red;  
8     public Color normalColor = Color.green;  
9  
10    private Renderer camRenderer;  
11  
12    void Start()  
13    {  
14        camRenderer = GetComponentInChildren<Renderer>(); // Pour  
15        changer sa couleur  
16    }  
17  
18    void Update()  
19    {  
20        // Lancer un rayon depuis la camera vers l'avant  
21        Ray ray = new Ray(transform.position, transform.forward);  
22        RaycastHit hitInfo;  
23  
24        // Debug : Voir le rayon dans la Scene View  
25        Debug.DrawRay(ray.origin, ray.direction * viewDistance, Color.  
26        yellow);  
27  
28        // Test de collision  
29        if (Physics.Raycast(ray, out hitInfo, viewDistance,   
30        obstaclesLayer))  
31        {  
32            Debug.Log("Je vois : " + hitInfo.collider.name);  
33  
34            if (hitInfo.collider.CompareTag("Player"))  
35            {  
36                // J'ai vu le joueur !  
37                ChangeColor(alertColor);  
38            }  
39            else  
40            {  
41                // J'ai vu un mur ou autre chose  
42                ChangeColor(normalColor);  
43            }  
44        }  
45    }  
46 }
```

```

40         }
41     }
42     else
43     {
44         // Je ne vois rien
45         ChangeColor(normalColor);
46     }
47 }
48
49 void ChangeColor(Color c)
50 {
51     if (camRenderer != null) camRenderer.material.color = c;
52 }
53 }

```

0.3.2 2. Configuration

1. Taggez votre Cube “Player” avec le Tag **Player** (Inspector > Tag > Add Tag > Player).
2. Ajoutez des **Murs** (Cubes allongés) dans la scène pour vous cacher.
3. Assignez le Layer “Default” à **obstaclesLayer** dans le script (ou créez un Layer spécifique).

Test : Passez devant la caméra → Elle devient Rouge. Cachez-vous derrière un mur → Elle redevient Verte.

C’est le principe de base de l’**Occlusion** en AR : Si un objet réel (Mur) est devant l’objet virtuel (Player), on ne doit pas le voir. Ici, c’est l’inverse (la caméra virtuelle voit le joueur virtuel), mais la mathématique est la même.

0.4 Partie 3 : Interaction par le Regard (Gaze)

En VR sans manettes, on utilise souvent le regard (“Regarder fixement un bouton pour cliquer”).

1. Créez un script **GazeButton.cs**.
2. Créez une Sphère qui servira de bouton.

```

1 using UnityEngine;
2
3 public class GazeButton : MonoBehaviour
4 {
5     public float activationTime = 2.0f; // Temps a regarder pour activer
6     private float timer = 0f;
7     private bool isBeingLookedAt = false;
8
9     public void OnLookStart()
10    {
11        isBeingLookedAt = true;
12    }
13
14    public void OnLookEnd()
15    {
16        isBeingLookedAt = false;
17        timer = 0f; // Reset
18        GetComponent<Renderer>().material.color = Color.white;
19    }
20
21    void Update()

```

```
22 {
23     if (isBeingLookedAt)
24     {
25         timer += Time.deltaTime;
26
27         // Changement progressif de couleur (Feedback visuel)
28         float ratio = timer / activationTime;
29         GetComponent<Renderer>().material.color = Color.Lerp(Color.
white, Color.green, ratio);
30
31         if (timer >= activationTime)
32         {
33             Debug.Log("BOUTON ACTIVE !");
34             timer = 0f;
35             // Faire une action ici (Ouvrir porte, etc.)
36         }
37     }
38 }
39 }
```

3. Modifiez votre script de caméra (ou créez un script sur la Main Camera du joueur) pour appeler OnLookStart et OnLookEnd.

— *Indice* : Utilisez `hitInfo.collider.GetComponent<GazeButton>()`.